



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251214
Nama Lengkap	Raffiel Gideon Nyolo Nyolo
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Pengertian Rekursif

Fungsi rekursif, juga disebut sebagai "fungsi yang memanggil dirinya sendiri", adalah fungsi yang berisi dirinya sendiri atau fungsi yang mendefinisikan dirinya sendiri. Fungsi rekursif adalah fungsi matematis yang berulang yang memiliki pola yang terstruktur, tetapi biasanya diperlukan perhatian agar fungsi ini dapat berhenti dan tidak menghabiskan memori. Fungsi rekursif harus digunakan dengan hati-hati karena memiliki pola yang terstruktur.

Untuk fungsi rekursif, ada dua blok penting: blok yang berfungsi sebagai titik berhenti dari proses rekursif dan blok yang memanggil dirinya sendiri. Fungsi ini akan berlanjut sampai kondisi berhenti terpenuhi. Rekursif terdiri dari dua bagian:

- **Base Case** adalah bagian dimana penentu bahwa fungsi rekursif itu berhenti.
- **Rekursif Case** adalah bagian dimana terdapat statement yang akan terus diulang-terus menerus hingga mencapai *Base Case*.

B. Kelebihan dan Kekurangan Fungsi Rekursif

Beberapa keunggulan fungsi rekursif adalah sebagai berikut:

1. Kode program lebih singkat dan elegan.

2. Masalah kompleks dapat di breakdown menjadi sub masalah yang lebih kecil di dalam rekursif.

Sedangkan kelemahan fungsi rekursif adalah:

1. Memakan memori yang lebih besar karena setiap kali bagian dirinya dipanggil maka dibutuhkan sejumlah ruang memori tambahan.
2. Mengorbankan efisiensi dan kecepatan.
3. Fungsi rekursif sulit dilakukan debugging dan kadang sulit dimengerti.

C. Bentuk Umum dan Studi Kasus

Bentuk umum fungsi rekursif pada Python:

```
def function_name(parameter_list):  
    ...  
    function_name(...)   
    ...
```

Sebenarnya, setiap fungsi rekursif pasti memiliki solusi iteratifnya. Contoh kasus faktorial berikut adalah sebagai berikut: Faktorial adalah menghitung perkalian deret angka $1 \times 2 \times 3 \times \dots \times n$.

Algoritma yang digunakan untuk menghitung faktorial adalah sebagai berikut:

1. Tanyakan n
2. Siapkan variabel total untuk menampung hasil perkalian dan set nilai awal dengan 0.
3. Loop dari $i = 1$ hingga n untuk mengerjakan:
4. $total *= i$
5. Tampilkan total

```
def faktorial(n):
    if n==0 or n==1:
        return 1
    else:
        return faktorial (n - 1) * n
print (faktorial (4))
```

Output: 24.

```
1.
2. calc_factorial(4)           # 1st call with 4
3. 4 * calc_factorial(3)       # 2nd call with 3
4. 4 * 3 * calc_factorial(2)   # 3rd call with 2
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
6. 4 * 3 * 2 * 1               # return from 4th call as number=1
7. 4 * 3 * 2                   # return from 3rd call
8. 4 * 6                       # return from 2nd call
9. 24                          # return from 1st call
```

Gambar 13.1: Proses perhitungan faktorial rekursif

Pseudocode (recursive):

```
function factorial is:
input: integer  $n$  such that  $n \geq 0$ 
output:  $[n \times (n-1) \times (n-2) \times \dots \times 1]$ 

    1. if  $n$  is 0, return 1
    2. otherwise, return  $[ n \times \text{factorial}(n-1) ]$ 

end factorial
```

Gambar 13.2: Pseudocode rekursif faktorial

Contoh studi kasus selanjutnya adalah fibonacci.

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
print(fibonacci(10))
```

output: 55

Kode ini mendefinisikan fungsi rekursif untuk menghitung bilangan Fibonacci ke-n, di mana setiap bilangan adalah jumlah dari dua bilangan sebelumnya, dimulai dari 0 dan 1.

Penjelasan:

1. **Base Case**-nya, jika $n == 0$, fungsi mengembalikan 0. Jika $n == 1$, fungsi mengembalikan 1. Kedua kasus ini diperlukan untuk menghentikan rekursi.
2. **Recursive Case**-nya, fungsi memanggil dirinya sendiri dua kali dengan pengurangan n (yaitu, $\text{fibonacci}(n-1)$ dan $\text{fibonacci}(n-2)$), menjumlahkan hasil dari panggilan-panggilan ini.

Contoh studi kasus selanjutnya yaitu hitung mundur.

```
def hitung_mundur(n):  
    if n <= 0:  
        print("Done!")  
    else:  
        print(n)  
        hitung_mundur(n - 1)  
  
hitung_mundur(5)
```

Output:

5

4

3

2

1

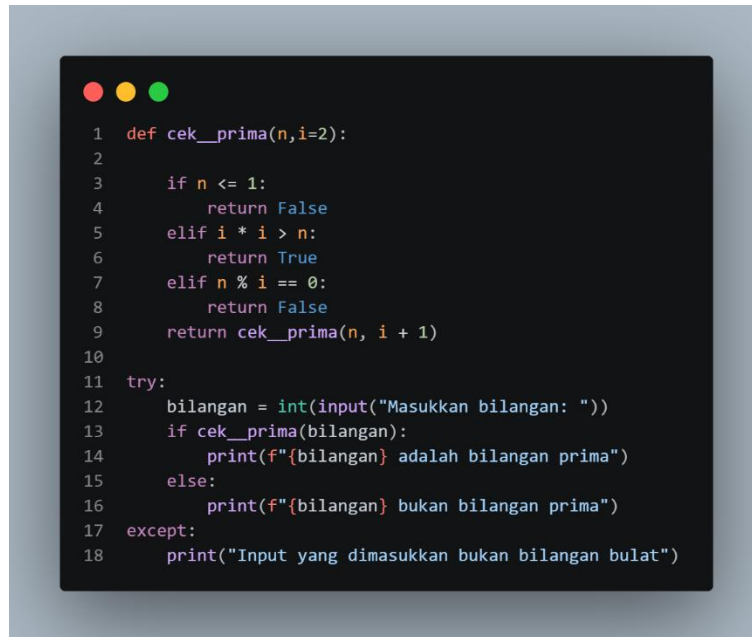
Done!

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github: <https://github.com/fieliel-star/tugas-prakAlpro-minggu1-71251214.git>

SOAL 1



```
1 def cek_prima(n,i=2):
2
3     if n <= 1:
4         return False
5     elif i * i > n:
6         return True
7     elif n % i == 0:
8         return False
9     return cek_prima(n, i + 1)
10
11 try:
12     bilangan = int(input("Masukkan bilangan: "))
13     if cek_prima(bilangan):
14         print(f"{bilangan} adalah bilangan prima")
15     else:
16         print(f"{bilangan} bukan bilangan prima")
17 except:
18     print("Input yang dimasukkan bukan bilangan bulat")
```

Penjelasan:

Kode Python tersebut berfungsi untuk memeriksa apakah suatu bilangan bulat yang dimasukkan pengguna adalah bilangan prima atau bukan menggunakan metode **rekursif**. Pada program utama, input pengguna dilindungi oleh blok try-except agar program tidak *crash* dan menampilkan pesan peringatan jika pengguna salah memasukkan karakter selain bilangan bulat.

Saat angka valid diterima, fungsi `cek_prima` akan mengujinya secara bertahap mulai dari pembagi ($i = 2$). Fungsi ini akan langsung mengembalikan nilai False jika angka kurang dari atau sama dengan 1 atau jika angka tersebut habis dibagi i . Sebaliknya, fungsi akan mengembalikan nilai True (bilangan prima) demi efisiensi jika kuadrat pembagi sudah melewati nilai angka yang diuji. Jika belum ada kondisi yang terpenuhi, fungsi akan memanggil dirinya sendiri kembali dengan menaikkan nilai pembagi sebesar 1 ($i + 1$) hingga didapatkan hasil akhir untuk dicetak ke layar.

Output:

```
PS D:\Semester_2\Praktikum
Masukkan bilangan: 5
5 adalah bilangan prima
PS D:\Semester_2\Praktikum
Masukkan bilangan: 12
12 bukan bilangan prima
```

SOAL 2

```
1 def cek_palindrom(nilai):
2
3     nilai_str = str(nilai)
4
5     if len(nilai_str) <= 1:
6         return True
7     else:
8         return nilai_str[0] == nilai_str[-1] and cek_palindrom(nilai_str[1:-1])
9
10
11 try:
12     nilai = int(input("Masukkan bilangan:"))
13     if cek_palindrom(nilai):
14         print("Nilai ini merupakan bilangan palindrom!")
15     else:
16         print("Nilai ini bukan merupakan bilangan palindrom!")
17 except :
18     print("Input yang dimasukkan bukan angka")
```

Penjelasan:

Kode Python tersebut berfungsi untuk memeriksa apakah suatu bilangan yang dimasukkan oleh pengguna merupakan bilangan palindrom (bilangan yang nilainya sama saat dibaca dari depan maupun belakang) menggunakan metode **rekursif**. Pada bagian utama, program menggunakan blok try-except untuk menangkap error, jika pengguna memasukkan karakter selain angka, program tidak akan *crash* melainkan menampilkan pesan peringatan. Setelah input angka valid diterima, angka tersebut diubah menjadi tipe data string di dalam fungsi `cek_palindrom` agar setiap karakternya bisa diakses berdasarkan indeks posisi.

Proses pengecekan dilakukan secara bertahap dengan membandingkan karakter pertama (`[0]`) dan karakter terakhir (`[-1]`) dari string tersebut. Jika kedua karakter ujung tersebut sama, fungsi akan memanggil dirinya sendiri kembali secara rekursif untuk memeriksa sisa karakter di

bagian tengahnya ([1:-1]). Proses ini berulang hingga menyisakan satu karakter atau string kosong ($\text{len} \leq 1$), yang menandakan bilangan tersebut terbukti palindrom (True). Namun, jika pada salah satu tahap pengecekan karakter ujung didapati tidak sama, fungsi akan langsung mengembalikan nilai False (bukan palindrom) untuk dicetak hasilnya ke layar.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_
Masukkan bilangan:404
Nilai ini merupakan bilangan palindrom!
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_
Masukkan bilangan:233
Nilai ini bukan merupakan bilangan palindrom!
```

SOAL 3

```
1  def deretGanjil(n):
2
3      if n == 1:
4          return 1
5      if n % 2 != 0:
6          return n + deretGanjil(n - 2)
7      else:
8          return deretGanjil(n - 1)
9
10 try:
11     nomor = int(input("Masukkan bilangan: "))
12     if nomor < 1:
13         print("Bilangan harus lebih besar atau sama dengan 1")
14     else:
15         hasil = deretGanjil(nomor)
16         print(f"Jumlah deret bilangan ganjil hingga {nomor} adalah {hasil}")
17 except:
18     print("Input yang dimasukkan bukan bilangan bulat")
```

Penjelasan:

Kode Python tersebut berfungsi untuk menghitung **jumlah total dari semua bilangan ganjil** mulai dari angka 1 hingga batas angka tertentu (n) yang dimasukkan oleh pengguna menggunakan metode **rekursif**. Program utama mengamankan input pengguna dengan blok try-except untuk mencegah *crash* jika dimasukkan karakter selain bilangan bulat, serta dilengkapi validasi untuk memastikan angka yang diinput tidak kurang dari 1. Setelah input valid diterima, program akan memanggil fungsi `deretGanjil` untuk memproses perhitungan tersebut.

Di dalam fungsi `deretGanjil`, sistem akan mengecek kondisi angka secara mundur. Jika nilai n sudah mencapai angka 1, fungsi langsung mengembalikan nilai 1 sebagai batas akhir rekursi (*base case*). Jika nilai n saat itu merupakan bilangan ganjil, fungsi akan menjumlahkan nilai n tersebut dengan hasil pemanggilan dirinya sendiri untuk angka ganjil sebelumnya ($n - 2$). Namun, jika nilai n yang diperiksa adalah bilangan genap, fungsi hanya akan memanggil dirinya sendiri kembali dengan menurunkan nilainya sebesar 1 ($n - 1$) tanpa menjumlahkannya, hingga seluruh bilangan ganjil dalam jangkauan tersebut selesai diakumulasikan dan hasilnya dicetak ke layar.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_r
Masukkan bilangan: 10
Jumlah deret bilangan ganjil hingga 10 adalah 25
```

SOAL 4

```
1 def digitRec(n):
2     if n == 0:
3         return 0, ""
4
5     digit = n % 10
6     sisa = n // 10
7
8     hasil, digitStr = digitRec(sisa)
9     hasil = hasil + digit
10
11     if digitStr == "":
12         digitStr = str(digit)
13     else:
14         digitStr = digitStr + " + " + str(digit)
15     return hasil, digitStr
16
17 try:
18     nomor = int(input("input bilangan: "))
19     if nomor < 0:
20         print("bilangan harus positif")
21     elif nomor == 0:
22         print("0 = 0")
23     else:
24         hasil, digitStr_output = digitRec(nomor)
25         print(f"{digitStr_output} = {hasil}")
26 except:
27     print("Input bukan bilangan bulat")
```

Penjelasan:

Kode Python tersebut berfungsi untuk memisahkan dan menghitung jumlah total dari setiap digit suatu bilangan bulat positif menggunakan metode **rekursif**. Program utama menggunakan blok try-except untuk mencegah *crash* dari input non-angka, serta memastikan bilangan yang dimasukkan bernilai positif. Setelah input valid diterima, fungsi digitRec akan memproses angka tersebut dan mengembalikan dua hasil sekaligus: total penjumlahan digit dan teks rincian prosesnya.

Di dalam fungsi digitRec, digit terakhir diambil menggunakan operasi modulus ($n \% 10$) dan sisa angka di depannya dipotong menggunakan pembagian bulat ($n // 10$). Fungsi ini terus memanggil dirinya sendiri secara rekursif hingga angka habis ($n == 0$). Saat proses rekursi selesai dan kembali naik, fungsi akan mengakumulasi nilai setiap digit ke dalam variabel jumlah serta merangkai digit-digit tersebut dengan pemisah tanda tambah (+) untuk dicetak sebagai hasil akhir di layar.

Output:

```
PS D:\Semester_2\Praktikum
nerFile.py"
input bilangan: 234
2 + 3 + 4 = 9
PS D:\Semester_2\Praktikum
nerFile.py"
input bilangan: 567
5 + 6 + 7 = 18
```

SOAL 5

```
1 def kombinasi(n, k):
2     if n < k:
3         print("Nilai dari n harus lebih besar atau sama dengan nilai k")
4         return None
5     elif k == 0 or k == n:
6         return 1
7     return kombinasi(n - 1, k - 1) + kombinasi(n - 1, k)
8
9 n, k = map(int, input("Masukkan nilai n dan k (pisahkan dengan koma): ").split(","))
10 hasil = kombinasi(n, k)
11 if hasil is not None:
12     print(f"Kombinasi C({n}, {k}) adalah {hasil}")
```

Penjelasan:

Kode Python tersebut berfungsi untuk menghitung nilai kombinasi $C(n, k)$ menggunakan metode **rekursif** berdasarkan identitas Segitiga Pascal. Pada program utama, pengguna diminta memasukkan nilai n dan k secara bersamaan dengan pemisah koma, yang kemudian dikonversi menjadi bilangan bulat untuk diproses oleh fungsi kombinasi.

Di dalam fungsi kombinasi, sistem pertama-tama memvalidasi bahwa nilai n tidak boleh lebih kecil dari k . Selanjutnya, fungsi menggunakan dua kondisi batas (*base cases*), jika $k = 0$ atau $k = n$, fungsi akan langsung mengembalikan nilai 1. Jika kondisi batas belum tercapai, fungsi akan memecah perhitungan secara rekursif dengan menjumlahkan dua pemanggilan fungsi baru, yaitu $kombinasi(n - 1, k - 1)$ dan $kombinasi(n - 1, k)$, hingga seluruh percabangan selesai dihitung dan hasil akhirnya dicetak ke layar.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_
Masukkan nilai n dan k (pisahkan dengan koma): 7,4
Kombinasi C(7, 4) adalah 35
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_
Masukkan nilai n dan k (pisahkan dengan koma): 2,4
Nilai dari n harus lebih besar atau sama dengan nilai k
```